

Module 8

Automating Cloud Resources

Module Objectives

- Explore automation and orchestration to manage code-based deployments.
- Use DevOps tools to provide Continuous Integration/Continuous Deployment management.

Lesson 8.1

Deploy and Configure Cloud Resources Using Code

Cloud+ CV0-004 Certification Exam Objective:

- 2.4 Given a scenario, use code to deploy and configure cloud resources.

Deploy and Configure Cloud Resources Using Code

- Infrastructure as Code
- Configuration as Code
- Benefits of IaC and CaC Environments
- Versioning Practices
- Testing Procedures
- Documentation Practices
- Code Formats

Infrastructure as Code

- Traditional IT infrastructure vs. infrastructure as code
- Infrastructure components and their integration

Configuration as Code (CaC)

- Use cases
 - Web servers
 - Kubernetes clusters
- Tools

Benefits of IaC and CaC Environments

- Repeatability
- Drift detection

Versioning Practices

- Be consistent
- Advertise version features
- Explain the versioning format

Testing Procedures

- Unit
- Integration
- System
- Acceptance

Documentation Practices

- Importance
- Common issues
- Best practices

Code Formats

- Yet Another Markup Language (YAML) files
- JavaScript Object Notation (JSON) files

Lesson 8.2

Continuous Integration Continuous Deployment Pipelines

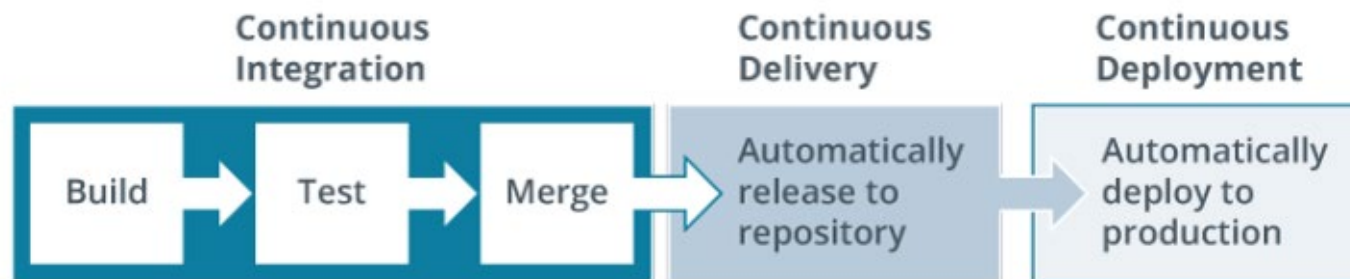
Cloud+ CV0-004 Certification Exam Objective:

- 5.2 Explain concepts related to continuous integration/continuous deployment (CI/CD) pipelines.

Continuous Integration Continuous Deployment Pipelines

- Continuous Integration Continuous Deployment
- Automation Practices
- Orchestration Practices
- Workflow Tasks
- Deployment Artifacts
- Code Repositories
- Secure Code Management

Continuous Integration Continuous Deployment (CI/CD)



CI/CD model featuring automation.

Automation Practices

- Routine operations
- Deployments and updates
- Scaling
- Shutdowns and restarts
- Create internal application programming interfaces (APIs)
- Troubleshoot automation issues

Orchestration Practices

- Automation concepts
- Orchestration concepts
- Orchestration workflow
- Manage orchestration sequencing

Workflow Tasks

- Code development
- Code integration
- Code testing
- Explain code repositories
- Explain code deployment
- Troubleshoot orchestration issues

Deployment Artifacts

- Virtual machine images
- Container images
- Application packages
- Bundle files using tar and zip
- Flat files

Code Repositories

- Public repositories
- Private repositories

Secure Code Management

- No hardcoded passwords
- Key-based authentication
- Password vaults
- Use of individual service accounts
- Adopt a secure coding standard
- Principle of least privilege

Lesson 8.3

Tools Used in DevOps Environments

Cloud+ CV0-004 Certification Exam Objective:

- 5.4 Explain the importance of tools used in DevOps Environments.

Tools Used in DevOps Environments

- DevOps Tools
- Ansible Orchestration
- Ansible Playbooks
- Docker Container Management
- Kubernetes Container Orchestration
- Elasticsearch, Logstash, and Kibana Stack
- Grafana Visualization Tool
- Jenkins CI/CD Tool
- Terraform IaC Tool

DevOps Tools

- Version control systems
- Continuous integration and continuous deployment
- Infrastructure as Code
- Configuration management
- Container management
- Monitoring and logging
- Integrated environment

Ansible Orchestration

- Capabilities
 - System provisioning
 - Application deployment
 - Configuration management
 - Integration with orchestration tools and practices
- Misconfigured or misapplied Ansible policies

Ansible Playbooks

- Tasks
- Plays
- Playbook

Docker Container Management

- Docker images
- Creating containers
- Viewing containers
- Container lifecycle
- Docker run command
- Working with container images
- Running the new image
- Viewing container logs
- Stopping containers

Kubernetes Container Orchestration

- Open-source container orchestration utility known for flexibility and easy integration
 - Key features
 - Capabilities
 - Benefits

Elasticsearch, Logstash, and Kibana Stack

- ELK stack benefits
- ELK stack deployment

Grafana Visualization Tool

- Visualization tool for creating interactive dashboards from compatible data sources
 - Key features
 - Data sources
 - Benefits

Jenkins CI/CD Tool

- CI/CD software development tool for build, test, and deploy phases
 - Key features
 - Platform
 - Integration
 - Workflow
 - Benefits

Terraform IaC Tool

- Orchestrating Infrastructure as Code (IaC) deployments using HashiCorp Configuration Language (HCL)
 - Key features
 - Benefits
 - Integration

Lesson 8.4

Source Control Concepts

Cloud+ CV0-004 Certification Exam Objective:

- 5.1 Explain source control concepts.

Source Control Concepts

- Source Control
- Version Management
- Git Concepts and Use Cases
- Git Repositories
- Git Commands
- Local Git Repository
- Git Branching and Collaboration
- Git Pull Request and Files
- Code Review
- GitHub Actions

Source Control

- Git
- Use cases
 - Ansible
 - Docker
 - Other tools

Version Management

- Git
 - Importance
 - Functions

Git Concepts and Use Cases

- Standardized configuration files
- Infrastructure as Code (IaC)
- Scripts
- Development projects

Git Repositories

- Getting started
- Core component
- Storage options
 - Local
 - Centralized

Git Commands

Subcommand	Description
config	Set options for a repository, Git users, or other global settings.
init	Create a Git repository or reinitialize an existing one.
clone	Create a working copy of an existing repository.
add	Add files to be tracked by the Git repository.
commit	Update the Git repository with your changes, creating a "snapshot" of that repository.

Subcommand	Description
status	Display the status of the repository.
branch	Manage branches (pointers to specific repository snapshots after committing changes).
merge	Integrate changes from one branch into a "main" branch.
rebase	Rewrites Git history for a cleaner, more linear merge history.
pull	Acquire and merge changes made to other repositories and branches into the local working copy.

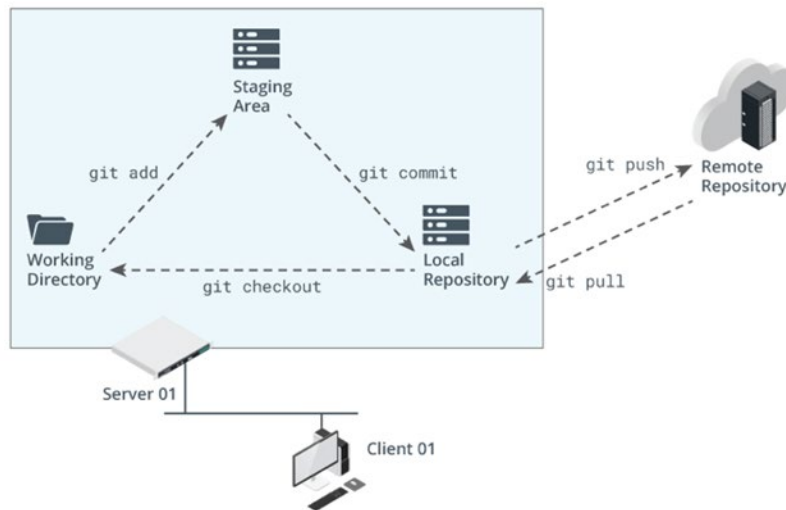
Git Commands (cont.)

Subcommand	Description
push	Upload a local working copy of a repository to a remote repository, such as a centralized repository.
log	Display the changes made to a local repository.
checkout	Switch to a specific branch to work with.
tag	Add a label to a repository's history, often to mark significant versions or releases.
show	Displays information on the current branch's status.

Local Git Repository

1. Configure global settings
2. Initialize repository
3. Add project files
4. Tag commits
5. Commit changes
6. Check status
7. Clone remote repository

Git Branching and Collaboration



- Branching
- Collaboration

An example Git process flow. Users operate within the working directory, the staging area, the local repository, and a remote repository with various git commands. (Images © 123RF.com.)

Git Pull Request and Files

- Git pull request
- The .gitignore file

Code Review

- Goals
 - Improve code quality.
 - Discover errors before production.
 - Suggest code improvements
- Two approaches
 - Static code review
 - Dynamic code review

GitHub Actions

- Definition
- Triggers
- Workflows
- Compatibility

Lesson 8.5

Scripting

Cloud+ CV0-004 Certification Exam Objective:

- 2.4 Given a scenario, use code to deploy and configure cloud resources.

Scripting

- Scripting Benefits
- Script Functions
- Script Variables
- Script Conditionals
- Script Loops
- Script Operators

Scripting Benefits

- Advantages
- Importance of script comments
- Script syntax
- Data types

Script Functions

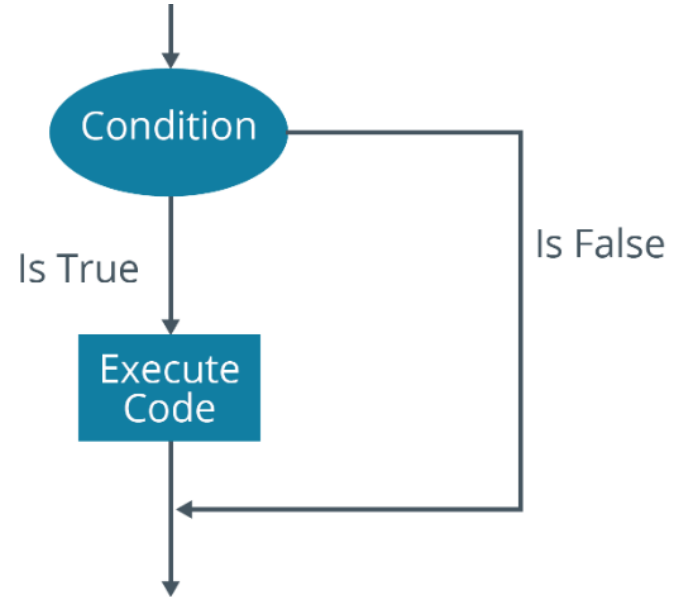
- Definition
- Benefits
- Syntax example (Bash)

Script Variables

- Definition
- Benefits
- Variable assignment
- Example (Bash)

Script Conditionals

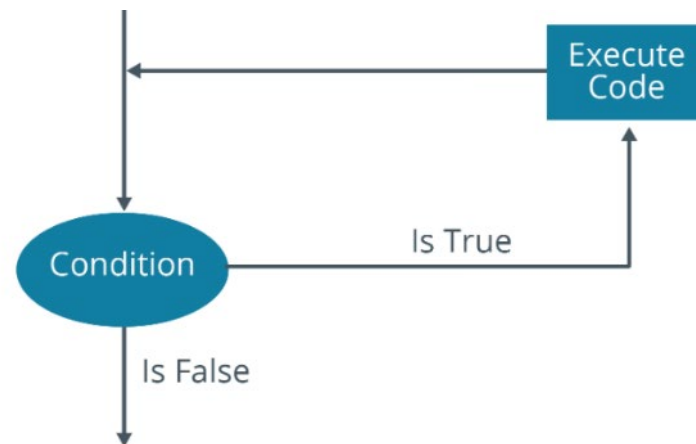
- The *if* conditional
- The *if...else* conditional
- The *test* command



Operation of a conditional statement, executing the code only if the condition is true.

Script Loops

- The *while* loop
- The *until* loop
- The *for* loop



► Description

This loop will repeat a series of commands or instructions as long as the conditions it depends on are true.

Script Operators

Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division

Boolean Operators

Operator	Description
&&	Logical AND (this and that)
	Logical OR (this or that)
!	Logical negation

Script Operators

Comparison and String Operators

Operator	Description
-eq	Equal to
-ne	Not equal to
-gt	Greater than
-ge	Greater than or equal to
-lt	Less than
le	Less than or qual to

Numeric Comparison Operators

Operator	Description
==	Equal to
!=	Not equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or qual to

Lesson 8.6

Integration of Systems

Cloud+ CV0-004 Certification Exam Objective:

- 5.3 Explain concepts related to integration of systems.

Integration of Systems

- Event-Driven Architectures
- Web Services as Components

Event-Driven Architectures

- Definition
- Example (e-commerce systems)
- Benefits
- Enabling technologies

Web Services as Components

- Understand REpresentational State Transfer (REST) design and APIs
- Simple Object Access Protocol (SOAP)
- Remote Procedure Call (RPC)
- WebSockets API
- GraphQL

Module Summary

- DevOps Practices: Essential for efficient, scalable, and accurate cloud management.
- Key Tools: Ansible, Docker, Kubernetes, Git, Grafana.
- Core Concepts: CI/CD, secure code repositories, version management.
- Benefits: Enhances health and efficiency of cloud environments.